Number System

Number System: It can be defined as the language of Numbers.

The most commonly used number systems are

		Radix
• Decimal Number system;	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}	10
• Binary Number system;	{0, 1}	2
• Octal "	; {0, 1, 2, 3, 4, 5, 6, 7}	8
• Hexa Decimal "	; {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}	16

Any radix to Decimal Number.

$$D = \sum_{i=-n}^{p-1} (d_i * r^i)$$

$p-1 \rightarrow$ No. of digits to the left of the radix point.
$d_i \rightarrow$ Value of the number.
$r_i \rightarrow$ Radix of the number system.
$n \rightarrow$ No. of digits to the right of the radix point.

Conversion

Binary to Decimal:

eg: $1010.01_{(2)}$

$$D = \sum_{i=-2}^{4-1=3} (d_i * 2^i)$$

$$D = (1\times2^{-2}) + (0\times2^{-1}) + (1\times2^{1}) + (0\times2^{2}) + (1\times2^{3})$$

$$= 0.25 + 0 + 0 + 2 + 0 + 8$$

$$= 10.25_{(10)}$$

Decimal to other number system:

eg1: $125_{(10)} = ?_{(2)}$

2	125
2	62 − 1
2	31 − 0
2	15 − 1
2	7 − 1
2	3 − 1
	1 − 1

$125_{(10)} = 1111101_{(2)}$

eg2: $0.7_{(10)} = ?_{(2)}$

$0.7 \times 2 = 1.4 \Rightarrow 1$
$0.4 \times 2 = 0.8 \Rightarrow 0$
$0.8 \times 2 = 1.6 \Rightarrow 1$
$0.6 \times 2 = 1.2 \Rightarrow 1$
$0.2 \times 2 = 0.4 \Rightarrow 0$
$0.4 \times 2 = 0.8 \Rightarrow 0$

$0.7_{(10)} = 0.101100_{(2)}$

$125.7_{(10)} = 1111101.101100_{(2)}$

Binary to Octal:

$(11101.011)_2 = (?)_8$

$\underbrace{011}_{3}\underbrace{101}_{5}.\underbrace{011}_{3}$

$(11101.011)_2 = (35.3)_8$

Binary to Hex:

$(11101.011)_2 = (?)_{16}$

11101.011

$\underbrace{0001}_{1}\ \underbrace{1101}_{D}\ .\ \underbrace{0110}_{6}$

$(11101.011)_2 = (1D.6)_{16}$

Octal to Binary

$(476.543)_8 = (?)_2$

- $\underbrace{4}_{100}\ \underbrace{7}_{111}\ \underbrace{6}_{110}\ .\ \underbrace{5}_{101}\ \underbrace{4}_{100}\ \underbrace{3}_{011}$

$(476.543)_8 = (100\,111\,110.\,101\,100\,011)_2.$

Hex to Binary

$(ADE.54C)_{16} = (?)_2.$

$\underbrace{A}_{1010}\ \underbrace{D}_{1101}\ \underbrace{E}_{1110}\ .\ \underbrace{5}_{0101}\ \underbrace{4}_{0100}\ \underbrace{C}_{1100}.$

$(ADE.54C)_{16} = (1010\,1101\,1110.\,0101\,0100\,1100)_2.$

Signed Numbers:

Schemes to represent the signed numbers
> Sign - Magnitude Representation
> Complement Representation.

Sign - Magnitude Representation:

- MSB is used as sign bit & the other bits represents the magnitude.

0 → for +ve
1 → for −ve

eg:
$1111_{(2)} = 15$ ⇒ unsigned Number
$\underline{0}1111_{(2)} = +15$ ⇒ signed +ve number.
$\underline{1}1111_{(2)} = -15$ ⇒ Signed −ve number.

- Range of the n bit signed magnitude integer:

$$-(2^{(n-1)}-1) \text{ to } +(2^{(n-1)}-1)$$

- Short comings:
  - Complex circuits   00000 +0
  - Two representations for 0   10000 -0

Complement Representation:

- 1's complement - Complementing all bits

eg: 1111(2) +15(10)
0000(2) -15(10)

- 2's complement - Add 1 to 1's complement

eg: 1111(2) = +15(10)
0001(2) = -15(10)

- Easy method to get 2's complement.

1011010(2)
0100101(2) => 1'complement
+1
0100110(2) => 2'complement

Start from LSB and use the same digits until you come across '1' and complement the remaining bits.

2's Complement for Binary Arithmetic.

Addition

7 => 0111
+4 => +0100
11 => 01011

Subtraction

4 => 0100
+ (-7) => + 1001
-3 => 1101

2's complement of 1101 is 0011

Key Points
- For sub, take 2's complement of the -ve number and add it with the other number.
- Carry 1 indicates positive number.

- Carry 0 indicates negative number.

Complement Number System.

System	Base	Possible complement systems.
Binary	2	1's comp & 2's comp.
Decimal	10	9's comp & 10's comp
Octal	8	7's comp & 8's comp
Hexa	16	15's comp & 16's comp.
Unknown	n	(n-1)'s comp & n's comp.

- n's complement of a number can be obtained by adding 1 to the (n-1)'s complement.
- (n-1)'s complement of a 'p' digit number can be obtained by subtracting the number from the maximum 'p' digit number.

Arithmetic using complement system.

$34_{(8)} - 27_{(8)}$

8's comp of $27_{(8)}$ is $(77-27)+1 = 51$

$2A_{(16)} - 4(C)_{16}$.

16's comp of $4C_{(16)}$ is $(FF-4C)+1 = B4$.

Codes :

- Symbolic representation.
- Types of codes.
  - Weighted binary codes
  - Non-weighted binary codes
  - Alphanumeric codes
  - Error detecting codes

Weighted Binary codes.

Decimal	8421	5421	2421	
0	0000	0000	0000	
1	0001	0001	0001	
2	0010	0010	0010	
3	0011	0011	0011	BCD code
4	0100	0100	0100	
5	0101	1000	1011	
6	0110	1001	1100	
7	0111	1010	1101	
8	1000	1011	1110	
9	1001	1100	1111	

BCD code for $874_{(10)}$

$874 = 1000\ 0111\ 0100_{(BCD)}$

Non-weighted codes

• Non-weighted codes are not positional weighted

eg: Excess-3 code gray code.

Gray code.

• The no. of bit changes is only one btn 2 consecutive numbers.

• Used in low power applications

Decimal	Binary	Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100

9 1001 1101

Binary and gray code conversion.

- Binary to gray code conversion.
[10110]$_2$ to gray code

```
1       0       1       1       0
1  XOR  1  xor  1  XOR  0  xor  1
```

$10110_{(2)} = 11101_{(gray)}$

- Gray to Binary code conversion.
[10110]gray to binary code.

```
1  xor  0       1       1       0
1       1       0       1       1
```